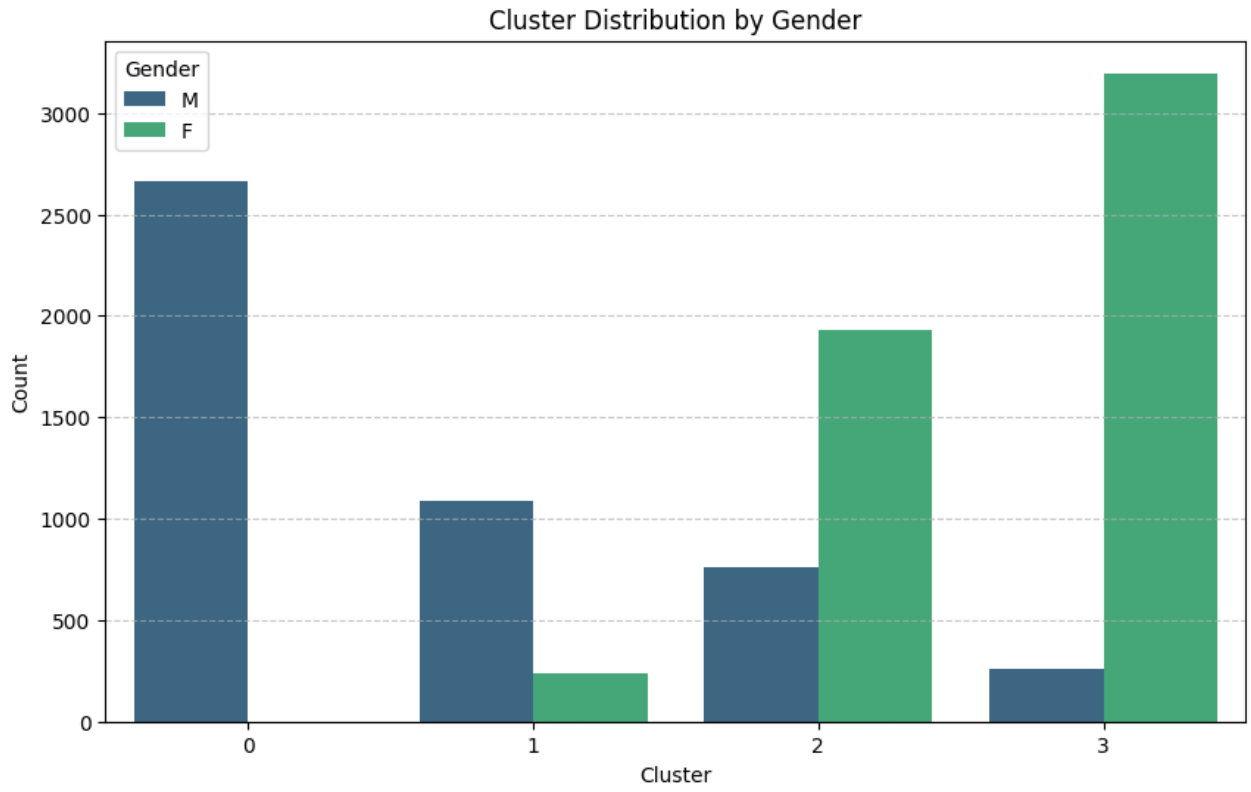
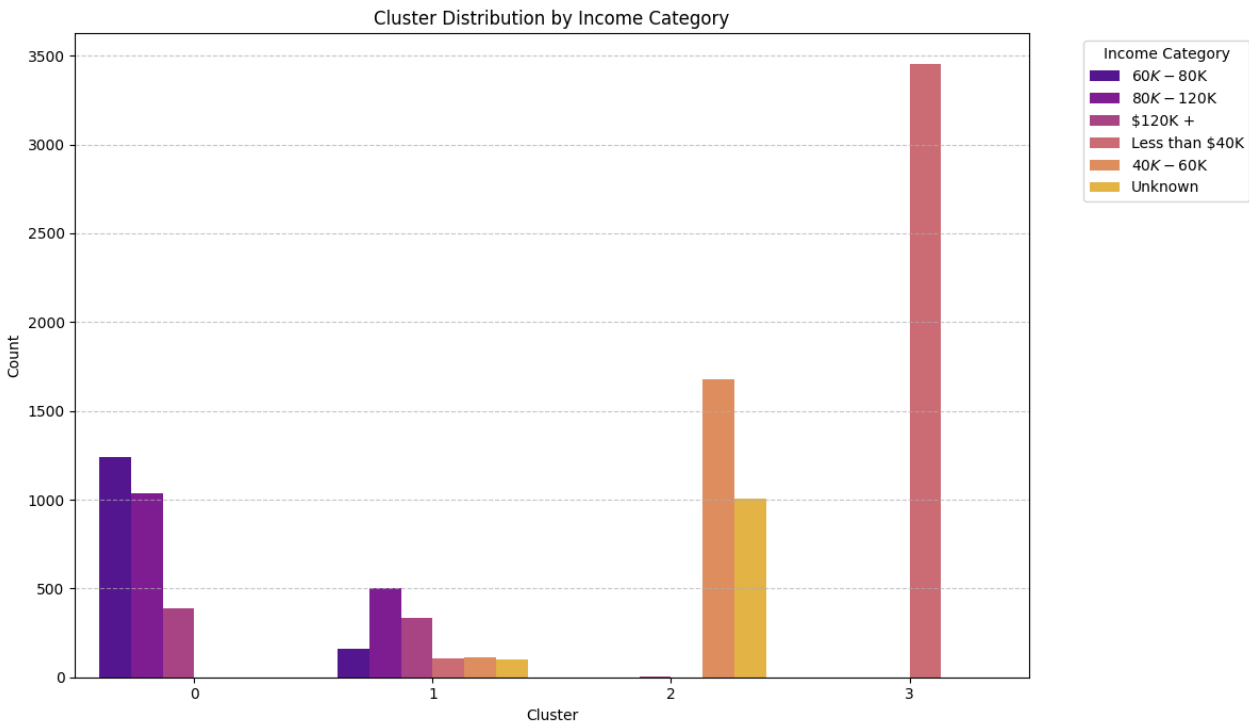
Fairness Audit: Cluster Distribution by Demographic Features

```
plt.figure(figsize=(10, 6))
sns.countplot(data=df_processed, x='Cluster', hue='Gender', palette='
plt.title('Cluster Distribution by Gender')
plt.xlabel('Cluster')
plt.ylabel('Count')
plt.legend(title='Gender')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



```
plt.figure(figsize=(12, 7))
sns.countplot(data=df_processed, x='Cluster', hue='Income_Category', p
plt.title('Cluster Distribution by Income Category')
plt.xlabel('Cluster')
plt.ylabel('Count')
plt.legend(title='Income Category', bbox_to_anchor=(1.05, 1), loc='up
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



Model Interpretability: SHAP Plot for KMeans

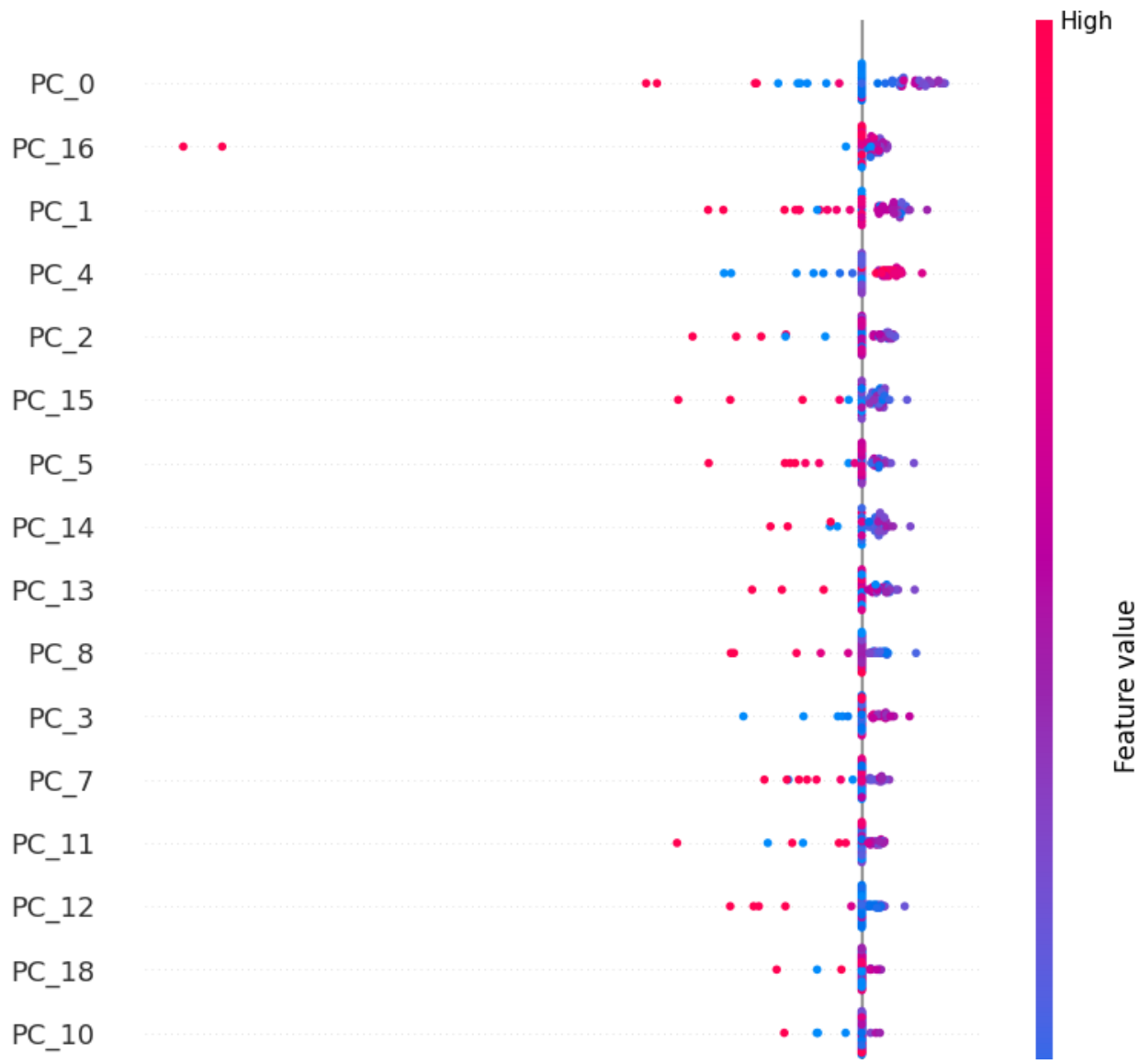
```
# Install SHAP library if not already installed
%pip install shap
```

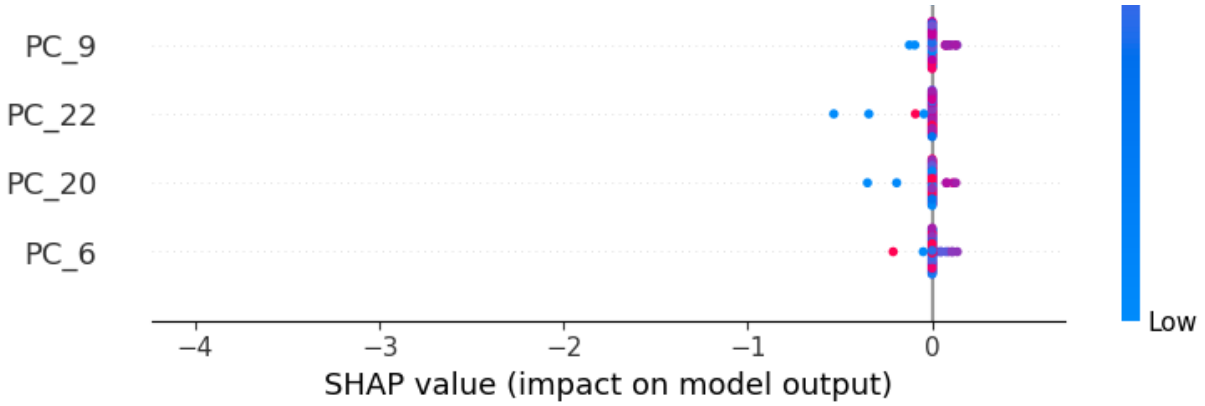
```
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/d
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dis
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3
Requirement already satisfied: numba in /usr/local/lib/python3.12/dist-
Requirement already satisfied: llvmlite in /usr/local/lib/python3.12/d
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.1
Requirement already satisfied: typing-extensions in /usr/local/lib/pytl
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lil
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/p
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/d
```

```
import shap

# The KMeans model predicts clusters, so we want to explain the clust
# For KMeans, SHAP can be used to understand feature importance for e
# We will use KernelExplainer and the cluster assignments as the mode

# Define a prediction function that returns cluster probabilities or
# For KMeans, we can define a 'predict' function that returns the dis
# SHAP values will then indicate how much each feature contributes to

def predict_proba_kmeans(X, kmeans_model):
    distances = kmeans_model.transform(X)
    # Inverse distances or just return distances for explanation
    # Smaller distance means higher 'probability' for that cluster
    return -distances # Negative distance so higher value means close

# Create a SHAP explainer
explainer = shap.KernelExplainer(lambda x: predict_proba_kmeans(x, km

# Sample the data once for both SHAP value calculation and plotting
data_for_shap_plot = shap.sample(df_pca, 50)

# Calculate SHAP values for the sampled data
```

```
shap_values = explainer.shap_values(data_for_shap_plot)

# Generate feature names for PCA components
pca_feature_names = [f"PC_{i}" for i in range(df_pca.shape[1])]

# Plot SHAP summary plot for the first cluster (you can iterate for a
print("SHAP summary plot for Cluster 0:")
# Correctly slice shap_values to get the SHAP values for the first ou
# Assuming shap_values structure is (n_samples, n_features, n_outputs
shap.summary_plot(shap_values[:, :, 0], data_for_shap_plot, feature_na

# Optionally, plot for other clusters or a different type of summary
# shap.summary_plot(shap_values, data_for_shap_plot, feature_names=pca
```

```
100%                                    50/50 [00:08<00:00,  6.47it/s]
SHAP summary plot for Cluster 0:
```




PC_9
PC_22
PC_20
PC_6
−4
−3
−2
−1
0
SHAP value (impact on model output)
Low